

ECE4760 Lab 3: PID control of a 1D helicopter

Stephen Barlett, Jamayne Gyimah-Danquah, Md Shad

November 13, 2025

1 Introduction

In this lab, we implemented a one-degree-of-freedom (1-DOF) helicopter system using the RP2040 microcontroller to explore feedback control principles and real-time embedded programming. The system consisted of a ducted fan mounted to the end of a lever arm, with the opposite end attached to a hinge that allowed rotation in a single plane. By varying the fan's thrust via pulse-width modulation (PWM), the arm could be lifted to a desired angle against gravity. The angular position was estimated using a complementary filter that fused gyroscope and accelerometer data from an onboard IMU communicated over an I²C interface, producing a smooth and reliable angle measurement suitable for feedback. A proportional, integral, derivative (PID) controller regulated the arm angle by continuously adjusting motor output to minimize the error between the measured and target positions. The proportional term (K_p) in our controller allowed for a correction proportional in regards to the current error as shown in our code, which produced a quick and rapid response; the integral term summed up past errors to prevent any steady-state offsets; and the derivative term allowed for predictions on future error based on the rate of change, which helped reduce any oscillations we saw. The controller ran at a fixed 1 kilohertz update rate within an interrupt service routine, ensuring consistent and deterministic timing for precise control. Task scheduling, including user input handling, peripheral initialization, and data visualization, was managed with protothreads, while the target angle and current beam angle were visualized in real time on a VGA display or oscilloscope. A serial interface allowed the user to dynamically adjust the desired angle and PID gains during operation.

The system demonstrated key embedded control concepts, including sensor calibration, signal filtering, electrical isolation of high-current motor circuits using optoisolators, and tuning of closed-loop response characteristics. Careful mechanical construction and circuit design were required to minimize noise and ensure stable operation. The motor drive stage was electrically isolated from the logic system to protect the RP2040, and both digital and analog low-pass filters were employed to suppress measurement noise that could destabilize the PID loop. The complementary filter effectively combined high-frequency gyro data with low-frequency accelerometer data to produce a smooth and reliable angle estimate suitable for feedback. Visualization of both the actual angle and the low-passed motor control signal enabled verification of system response and assisted in troubleshooting noise, bias, or instability issues.

The completed system demonstrated stable and responsive control across a wide range of setpoints, with the lever arm smoothly tracking user-specified angles. Through this project, we verified the theoretical behavior of PID control in a tangible electromechanical system while gaining practical experience with sensor fusion, real-time embedded computation, robust system integration on a resource-constrained microcontroller, PID tuning, and the challenges of minimizing noise and ensuring accurate measurements in electromechanical feedback systems.

2 Design and Testing

2.1 Hardware Overview

The hardware for this system consists of both electrical and mechanical components.

Electrically, the setup includes a Raspberry Pi Pico mounted on a solderless breadboard, a momentary push button for user input, a VGA extender PCB that interfaces the Pico with an external display, and a dedicated motor driver PCB for safely controlling a DC motor. The VGA extender board connects the Pico’s GPIO pins to a VGA D-sub connector through a resistive DAC network, while the motor PCB contains the MOSFET, optoisolator, flyback diode, and filtering components required to drive the motor under PWM control and protect the microcontroller.

Mechanically, the system uses a wooden board slab as the main structural base. This board is clamped to the table to keep the entire assembly rigid during operation. An acrylic arm is attached to the wooden board using screws and a nut, forming the rotating beam whose angle is controlled by the motor. The acrylic arm has a large hole at its top end that houses the body of the DC motor, allowing the motor shaft to be rigidly coupled to the arm so that the motor’s torque directly rotates the beam.

An IMU module is mounted at the base of the acrylic arm, close to the pivot point. Because the arm rotates in a vertical plane about a single axis, the IMU’s orientation is chosen so that:

- The accelerometer y and z axes primarily measure the projection of gravity as the arm tilts. As the arm angle θ changes, the gravitational acceleration vector decomposes into components along these axes (e.g., $a_y \propto \sin \theta$, $a_z \propto \cos \theta$), providing an angle-dependent measurement.
- The gyroscope x axis is aligned with the rotation axis of the arm, so the gyro’s x -axis output directly measures the angular velocity $\dot{\theta}$ of the beam.

Mounting the IMU at the pivot minimizes translational acceleration (which increases with distance from the pivot), so the accelerometer signal is dominated by gravity rather than inertial effects.

2.1.1 DC Motor Circuit

The design of this motor circuit was made purposely so it can reduced the chances of our microcontroller to be affected from any large $L \frac{di}{dt}$ voltage spikes that would be produced

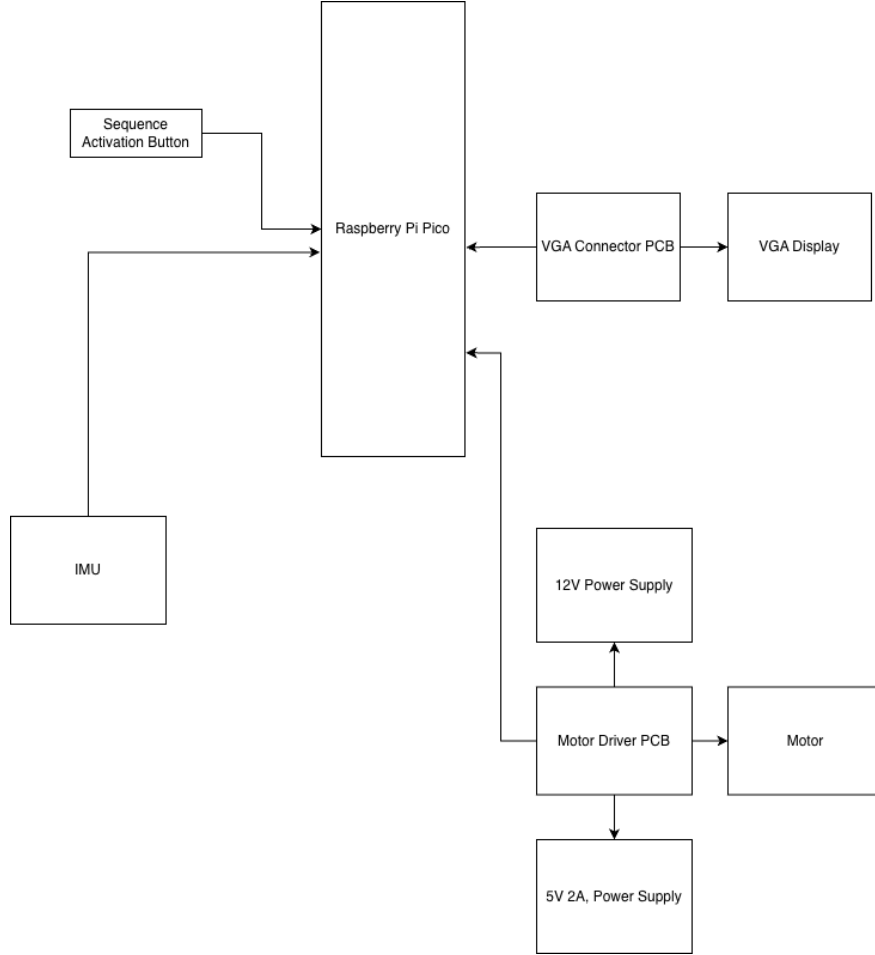


Figure 1: Electrical High Level Overview.

by the PWM-driven DC motor we were utilizing in our system. Electrically, the motor can be modeled as a series combination of a resistor (winding resistance), an inductor (the windings), and a voltage source representing the back-EMF generated by rotation. When the MOSFET switch turns on, the current in the motor windings ramps up. When the MOSFET turns off abruptly, the current is forced to drop from its previous value toward zero in a very short time. Because an inductor resists rapid changes in current, this large $\frac{di}{dt}$ causes a large induced voltage across the windings, resulting in a high-amplitude spike that must be safely redirected.

A 1N4001 flyback (snubber) diode is placed across the motor to provide a safe path for this inductive current during turn-off. When the motor terminal voltage reverses polarity due to the inductive kick, this causes the diode to move in forward-biased direction and conduct, letting the current to recirculate instead of producing a destructive voltage spike. In parallel with this diode, a $0.1 \mu\text{F}$ ceramic capacitor is added to shunt higher-frequency noise generated by commutation and fast PWM edges. We used ceramic capacitors here due their minimal equivalent series resistance and equivalent series inductance which make them an effective tool for filtering high-frequency transients. Thus these capacitors in our system provides a low-impedance path for very fast spikes and ringing that would otherwise couple



Figure 2: Mechanical Setup. See: Adams, "Mechanical Construction..."

into nearby circuitry.

To isolate the microcontroller from the noisy motor side, the gate drive is routed through a 4N35 optoisolator. Internally, the 4N35 contains an LED on the MCU side and a phototransistor on the motor side. When the MCU drives the corresponding GPIO pin high, current flows through the LED, which emits light and activates the phototransistor. On the output side, the phototransistor behaves like an NPN transistor whose "base" is driven by light; it pulls the gate-drive node low or high (depending on the surrounding circuitry) to switch the power MOSFET. Because the LED and phototransistor are separated by an optical gap, the two sides are galvanically isolated. As a result, ground noise, transients, or large voltage excursions on the motor/MOSFET side cannot directly propagate back into the MCU ground or I/O pins.

The 4N35 also exposes a physical base pin on the phototransistor. A 1 M Ω resistor from base to emitter is used to improve turn-off behavior: when the transistor is saturated, charge stored in the base region can slow its fall time. The resistor provides a discharge path for this stored charge, allowing the phototransistor to turn off more quickly when the LED is extinguished. The value of 1 M Ω is large enough that it does not significantly steal current from the light-driven base, but small enough to gently bleed off charge and sharpen the falling edge of the gate-drive signal.

The overall circuit includes input ports for a 12 V supply (used to drive the MOSFET gate through the isolated gate-drive network), a 5 V, 2 A supply from a wall adapter for the motor itself, a connector for the DC motor's + and - terminals, and a port for the PWM control signal coming from the MCU through the optoisolator. A physical kill switch is placed in series with the motor supply so that power to the motor can be cut instantly, stopping motion regardless of the PWM command and providing an additional safety layer.

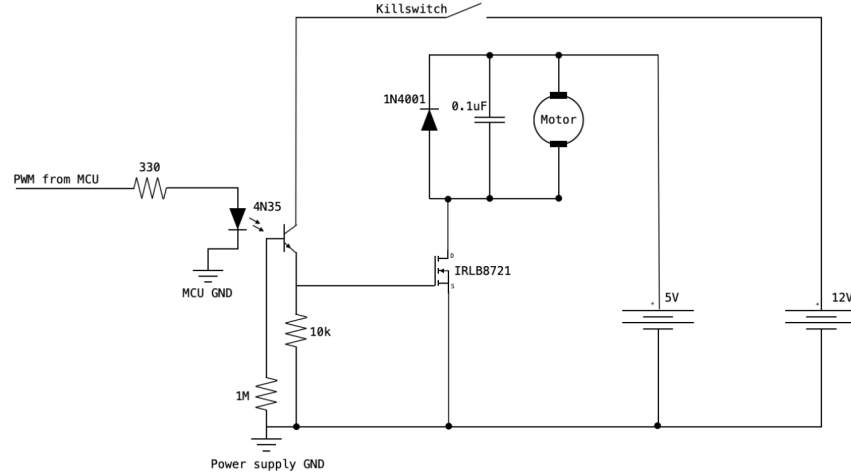


Figure 3: Motor Circuit. See: Adams, “DC Motor Circuit and PWM on RP2040.”

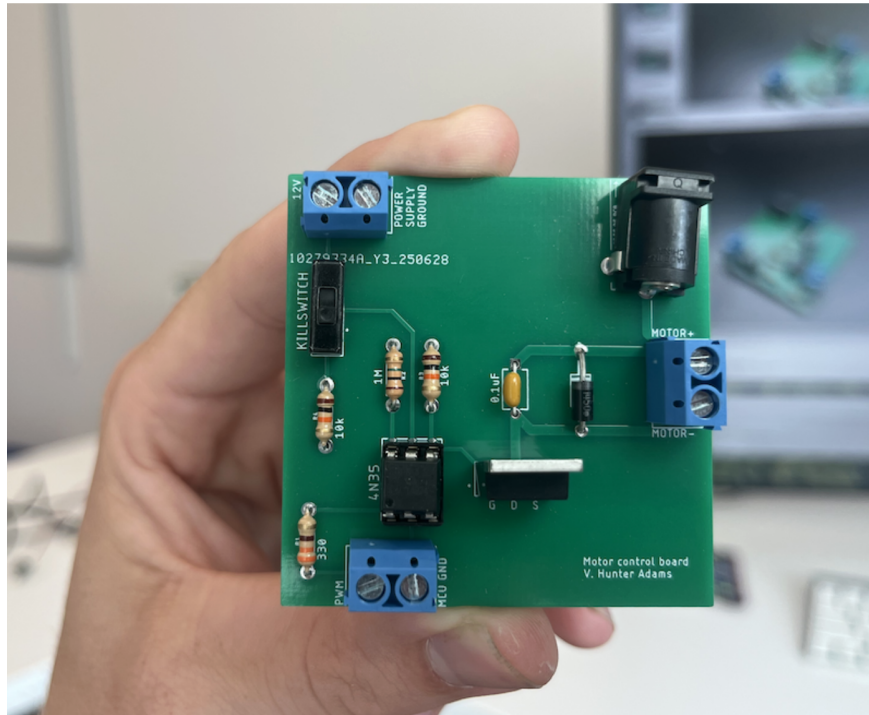


Figure 4: Motor Driver PCB. See: Adams, “DC Motor Circuit and PWM on RP2040.”

2.1.2 VGA Output Interface

To generate the VGA display, a VGA extender PCB was soldered to connect the Raspberry Pi Pico to a VGA D-sub connector through a simple resistive DAC network. The board contained through-hole components including resistors, header pins, and the D-sub connector itself. The following GPIO pins were used:

- GPIO 18 → Green (Lo bit) through a 470 Ω resistor

- GPIO 19 → Green (Hi bit) through a $330\ \Omega$ resistor
- GPIO 20 → Blue through a $330\ \Omega$ resistor
- GPIO 21 → Red through a $330\ \Omega$ resistor
- GPIO 16 → HSYNC
- GPIO 17 → VSYNC

The VGA standard expects an analog voltage of approximately 0–0.7 V into a $75\ \Omega$ load for each color channel. Since the Pico outputs 3.3 V digital signals, the series resistors form a voltage divider with the monitor’s $75\ \Omega$ input to scale the voltage down into the correct range.

Green is given two bits (Lo and Hi) because the human eye is most sensitive to green, so using two GPIOs for this channel allows four brightness levels for green (00, 01, 10, 11), improving perceived image quality. The $330\ \Omega$ and $470\ \Omega$ values are chosen so that:

- Each GPIO pin can safely drive the load without exceeding its current limits.
- The combined effect of the two resistors and the $75\ \Omega$ termination approximates a binary-weighted DAC, where the Hi bit contributes roughly twice as much brightness as the Lo bit and the maximum combined output is close to the 0.7 V VGA full-scale level.

Each resistor was soldered onto the PCB, and continuity testing was performed afterward to ensure that all connections from the Pico GPIOs to the VGA D-sub pins were electrically sound. Optical inspection under bright light was carried out to identify any potential cold solder joints, which could cause signal instability. Any dull or grainy joints were reflowed to ensure proper conductivity.

2.1.3 Button Input

A momentary push button was mounted on a breadboard and connected between GPIO 15 and GND. GPIO 15 was used as a digital input which contained an internal pull-up resistor, so when the button was not being pressed the input would logic high and it would logic low when the button was being pressed by an user demonstration a active-low configuration.

The button is used to start a predefined sequence of target angles for the motor-driven arm. Before the experiment begins (i.e., before time = 0), the button is held down so that the motor is disabled (PWM duty cycle set to zero) and the beam hangs vertically downward under gravity. As the button is let go at time = 0, the software puts the target angle to our horizontal value of (0°) and the motor uses PID control to drive the arm to this position. The desired angle is then updated in 5 s intervals:

- $t = 0$ s: button released → target angle set to 0° (horizontal)
- $t = 5$ s: target angle set to $\approx +30^\circ$ above horizontal
- $t = 10$ s: target angle set to $\approx -30^\circ$ below horizontal

- $t = 15$ s: target angle set back to 0° (horizontal)

To ensure reliable detection of the button release event, a software finite state machine (FSM) was implemented for debouncing. Mechanical push buttons do not switch cleanly; instead, their contacts “bounce”, producing several rapid transitions between high and low over a few milliseconds. If read directly, this would cause multiple false triggers.

2.1.4 Button Debouncing FSM

To obtain a clean digital signal from the mechanical push button, a software finite state machine (FSM) was implemented for debouncing. The FSM tracks both the logical level of the input and how long it has remained unchanged.

The FSM uses two key parameters:

- `DEBOUNCE_SAMPLES = 4` – the number of consecutive, identical samples required before a state transition is accepted.
- `same_cnt` – a counter that increments each time the input level remains the same between samples and is reset to 0 whenever the input changes.

At each periodic sampling of GPIO 15, the FSM moves through four states:

State 0 – Not Pressed The input is assumed to be unpressed (logic 1). The FSM waits for a falling edge (input $i \neq 1$). When a potential press is detected, it transitions to State 1 and resets `same_cnt = 0`.

State 1 – Maybe Pressed The FSM checks whether the input remains low (logic 0).

- If the input stays low, `same_cnt` is incremented.
- Once `same_cnt` reaches `DEBOUNCE_SAMPLES = 4`, the transition is accepted as a valid press, and the FSM enters State 2 (Pressed).
- If the input returns high before `same_cnt` reaches 4, the event is treated as bounce, `same_cnt` is reset, and the FSM returns to State 0.

State 2 – Pressed The button is considered firmly pressed while the input remains low ($i == 0$). If the input goes high ($i \neq 0$), indicating a possible release, the FSM transitions to State 3 and resets `same_cnt = 0`.

State 3 – Maybe Released The FSM now validates the release:

- If the input stays high, `same_cnt` increments.
- When `same_cnt` reaches `DEBOUNCE_SAMPLES = 4`, the release is accepted as valid, and the FSM returns to State 0 (Not Pressed), ready to detect the next press.
- If the input drops low again before reaching 4, the FSM treats this as bounce and goes back to State 2 (Pressed).

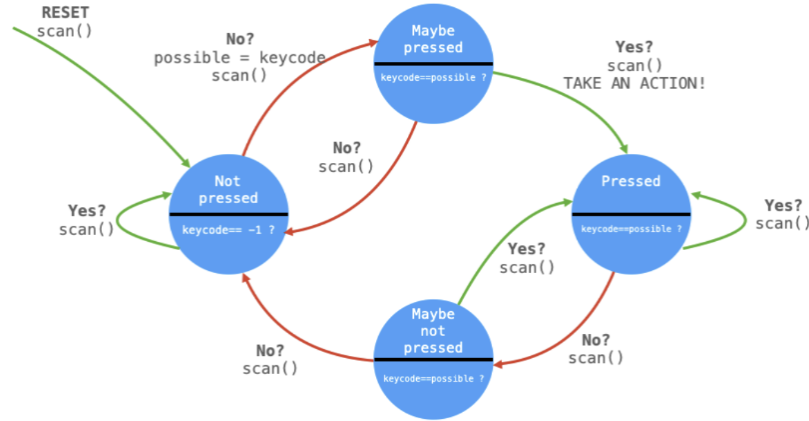


Figure 5: Debouncing FSM. See: Adams, “Interfacing a 3×4 Matrix Keypad with RP2040.”

2.1.5 Choice of Debounce Parameters

The value `DEBOUNCE_SAMPLES = 4` was chosen experimentally. With the sampling period used in the main control loop, this corresponds to a debounce window on the order of a few milliseconds, which is long enough to filter typical mechanical contact bounce but short enough to keep the button response feeling instantaneous to the user.

Smaller values (e.g., 1–2 samples) occasionally allowed spurious edges to slip through during fast bouncing, while larger values introduced a noticeable delay between the physical press/release and the logical event. Thus, 4 samples was selected as a practical compromise between noise immunity and responsiveness. The `same_cnt` variable enforces this threshold by only allowing state transitions after the input has remained stable for the required number of consecutive samples, ensuring that each physical press produces exactly one clean logical toggle.

2.1.6 Inertial Measurement Unit

An MPU6050 IMU was used to provide both 3-axis accelerometer and 3-axis gyroscope measurements for feedback control. This device communicates digitally over an I²C bus, which reduces wiring complexity and allows all sensor data to be read through two signal lines.

The IMU breakout exposes four connections: SCL (clock), SDA (data), 3.3 V power, and GND. Because the module was connected via a ribbon cable terminated in loose male wires, the wire colors had to be identified experimentally. A multimeter in continuity mode was used to probe from each ribbon wire back to the pins on the breakout:

- The wire continuous with the SCL pad was found to be yellow.
- The wire continuous with the SDA pad was blue.
- The red wire was connected to 3.3 V (VCC).

- The black wire was connected to GND.

These were then wired to the Raspberry Pi Pico as follows:

- GPIO 8 $\rightarrow$ MPU6050 SDA
- GPIO 9 $\rightarrow$ MPU6050 SCL
- 3.3 V $\rightarrow$ MPU6050 VCC
- RP2040 GND $\rightarrow$ MPU6050 GND

The IMU was placed onto the upper base of the acrylic arm facing the ceiling, with its axes oriented so that the accelerometer y and z axes measure the projection of gravity as the arm tilts, while the gyroscope x axis is aligned with the arm's rotation axis. In this configuration, the accelerometer provides an angle-dependent measurement and the gyro x -axis provides angular velocity. These signals are read over I²C and used as the primary sensor inputs to the PID control algorithm.

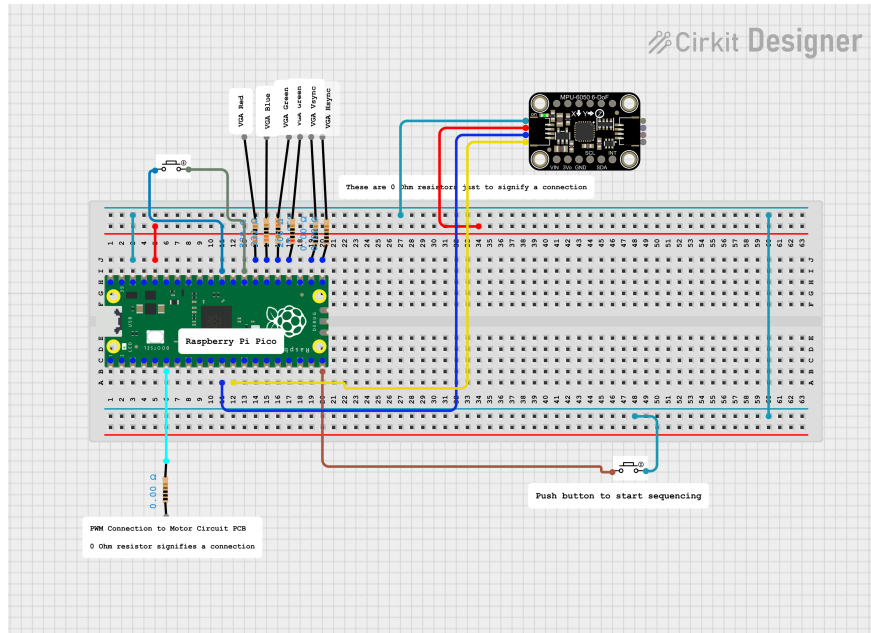


Figure 6: Breadboard Circuit Diagram.

3 Software Design

Our software design implements a robust PID controller which receives feedback by combining the gyro and accelerometer readings of an IMU in a software complementary filter. We controlled our motor by sending out a pulse-width modulated (PWM) signal to our motor control circuit utilizing the Pico's PWM hardware. This PWM signal can be manipulated by setting a global `control` variable, the output of our PID.

Our software also drives a VGA display, graphing the last 20 seconds of our desired angle, actual angle, and control value as well as displaying our current K_p , K_d , and K_i . Finally, a small portion of our software is dedicated to debouncing a button and driving our 20-second pre-programmed sequence.

3.1 PID Controller

Our PID controller is backed by a custom structure containing all of the relevant parameters and state information. Listing 1 shows the fields of our PID structure.

Listing 1: PID Struct in C code.

```

1 struct PID
2 {
3     fix15 Kp;
4     fix15 Ki;
5     fix15 Kd;
6     fix15 prev_err;
7     fix15 integral;
8     fix15 max_integral;
9     int max;
10    int min;
11 };

```

Our PID control loop runs 1000 times per second, aligned with our angle sampling and PWM wrap. We use hardware interrupts to ensure that our controller executes at a consistent rate, since the integral and derivative components depend on a fixed time constant τ .

On every loop iteration, we compute the error, update our accumulated error, and sample the gyroscope for $\Delta\theta$. We then compute our control output according to Equation 1.

$$C = \text{clamp}((K_p \cdot e_\Theta) + (K_i \cdot \Sigma_\Theta) + (K_d \cdot \Delta_\Theta), \text{min}, \text{max}) \quad (1)$$

3.2 Complementary Filter

We utilize a complementary filter to estimate the helicopter’s orientation using both accelerometer and gyroscope data. The gyroscope produces a clean signal that measures the rate of change in orientation but accumulates drift over time. The accelerometer measures the absolute orientation relative to Earth’s gravity but introduces substantial high-frequency noise.

To combine the strengths of both sensors, we high-pass filter the gyroscope data (removing long-term drift) and low-pass filter the accelerometer data (removing high-frequency noise). The resulting estimate provides both accuracy and responsiveness.

This is implemented by weighting the gyroscope estimate heavily (0.999) and the accelerometer estimate lightly (0.001):

$$\theta_{\text{filtered}} = 0.999(\theta_{\text{prev}} + \Delta\theta_{\text{gyro}}) + 0.001(\theta_{\text{accel}}) \quad (2)$$

This combination allows the for the gyro to dominate the angle estimation whenever it is significant and the accelerometer to correct any accumulated drift when the gyro settles. Figure 7 illustrates this process.

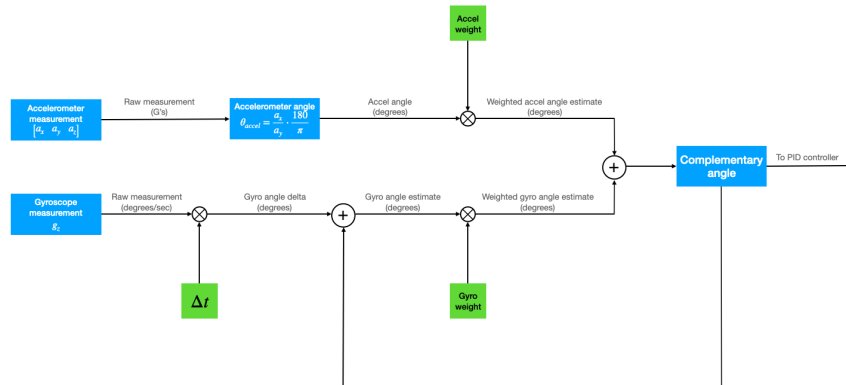


Figure 7: Block diagram of the complementary filter. See: Adams, “Complementary Filters.”

3.3 Sequence Playback

Our demo sequence is managed by a dedicated protothread that waits for a semaphore triggered by a debounced button press. Once signaled, the thread executes a four-stage sequence of setpoints (0° , $+30^\circ$, -30° and 0°), holding each intermediate value for 5 seconds.

For our demonstration, this sequence was hard-coded and returned the helicopter to horizontal when complete. Figure 8 shows a VGA trace of the desired and measured angles, along with the control signal, during the sequence playback. Saturation of the VGA color in this image was weak, so see Figure 9 for an enhanced visual.

4 Documentation and Lab Questions

4.1 Selecting PID Gains

Tuning the PID was an incremental process that started with our proportional K_p (≈ 30.0 to start), then our derivative K_d to reduce the oscillation around the desired angle. Once we found a stable setpoint for K_d (≈ 20.0 to start) we could increase K_p significantly as the derivative control protected the helicopter from snapping itself. After some iteration, we settled on a K_p of 200.0 and a K_d of 75. Finally, we added some K_i to eliminate steady state error. We found a stable starting point of around 2.5 for K_i .

One other parameter that we tweaked was the cap on our error accumulation I_{cap} . This cap prevents integral windup that would occur if the helicopter is held at a distance from the setpoint for an extended period of time. This value is complementary to our integral gain as we want the product of K_i and I_{cap} to be in the range from 0 to our maximum control value (so that the integral term can saturate the duty cycle when necessary). We found that a lower I_{cap} with a higher K_i resulted in decreased stability in addition to slower response

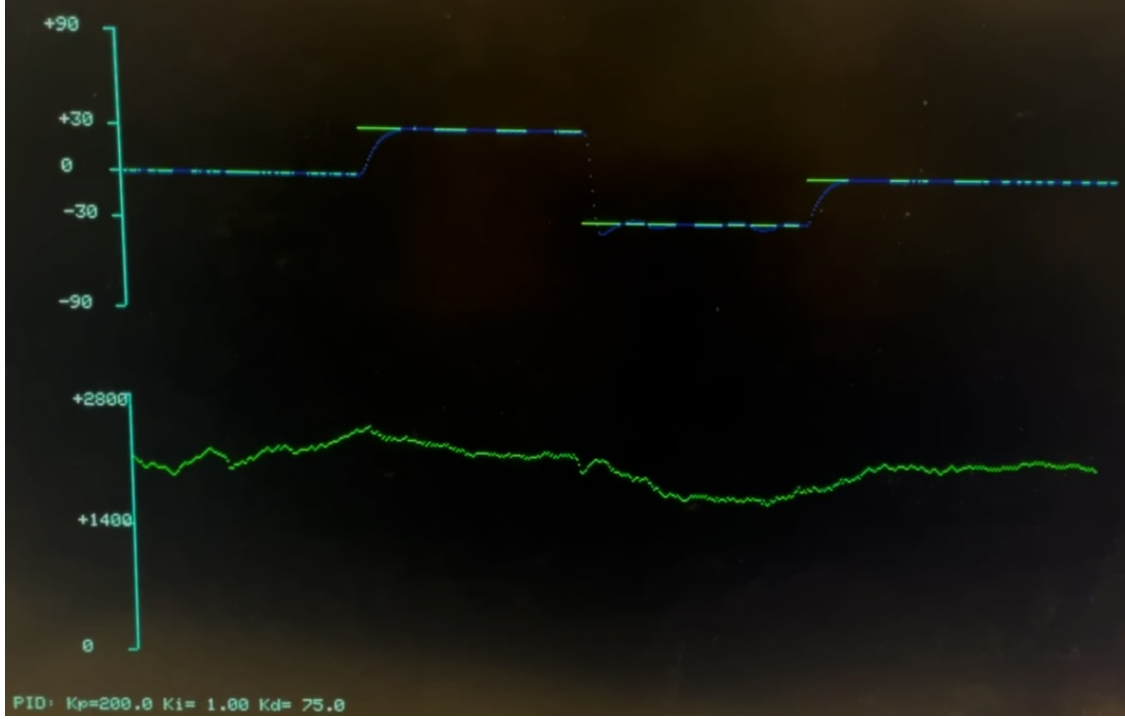


Figure 8: Screenshot of VGA output after sequence playback.

times. After some trial and error, we settled on an integral gain of 1.0 and an I_{cap} of 2800 as it yielded the best combination of snappy and stable.

4.2 Max/Min Stable Range

After extensive testing, we confirmed that our helicopter maintains stability for commanded angles between approximately -80° and $+80^\circ$ relative to horizontal. Beyond these limits, physical interference between the helicopter arm and mounting block prevents full range motion.

4.3 Settling Times

We logged the settling times for each angle transition in our 20-second sequence (Table 1). A tolerance of $\pm 6\%$ was used, as smaller tolerances often did not settle within the 5-second window. Visually, the helicopter's response appears snappy and accurate. The longest settling time occurred for the largest transition ($+30^\circ$ to -30°). We used two python scripts for the collection and analysis of this data. One script monitored a serial port that was receiving the complementary angle, desired angle, and control setpoint roughly every 3 ms and logged it to a txt file. This file was then processed by the second script, calculating exact settling times within our 6% tolerance. This method yielded very accurate, data backed, results - as well as the very nice chart seen in Figure 9.¹

¹AI Disclaimer: See our section on AI assistance for information about the creation of these python scripts.

Table 1: Settling times for 20-second sequence.

Transition	Change Time (s)	Settling Time (s)	Tolerance Band (deg)
$-90^\circ \text{ (off)} \rightarrow 0^\circ$	0.000	1.720	$[-0.6, 0.6]$
$0^\circ \rightarrow 30^\circ$	5.000	1.030	$[29.4, 30.6]$
$30^\circ \rightarrow -30^\circ$	10.000	1.250	$[-30.6, -29.4]$
$-30^\circ \rightarrow 0^\circ$	14.470	1.030	$[-0.6, 0.6]$

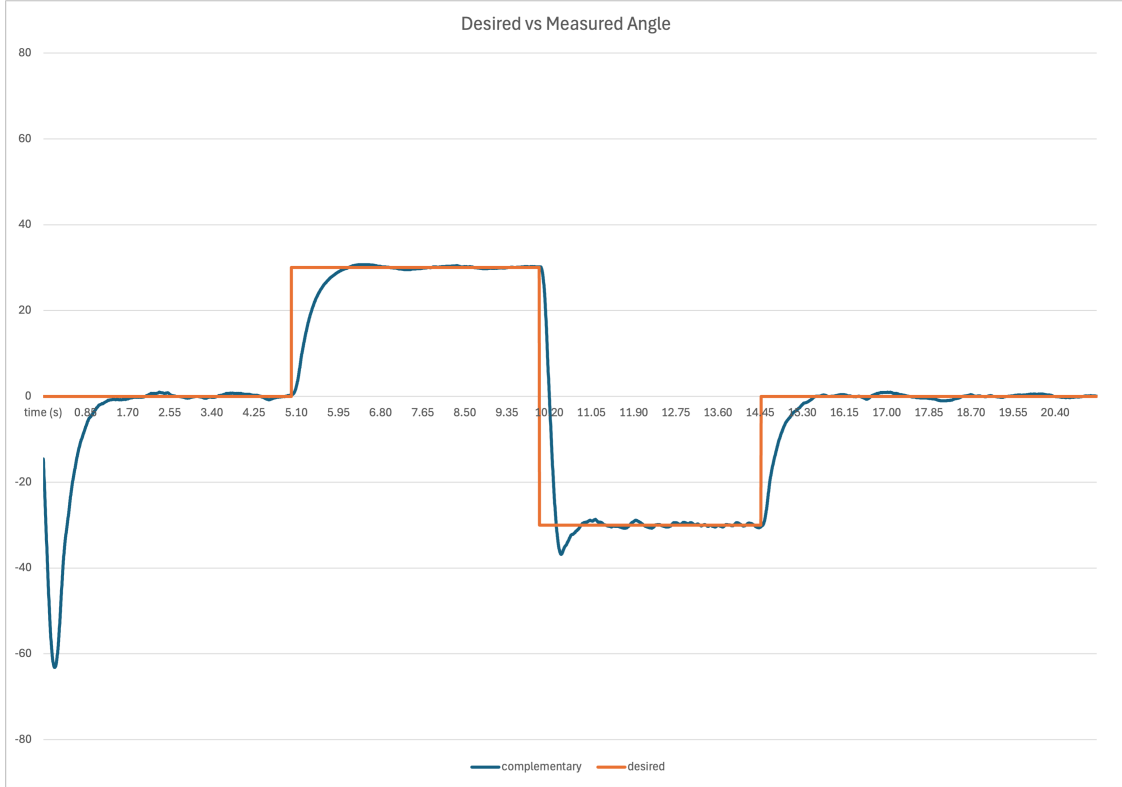


Figure 9: Desired and measured angles for 20-second sequence.

Testing

Most testing for this lab was performed by direct interaction with the RP2040 microcontroller and its connected peripherals, combining hardware inspection with targeted software diagnostics. Since the objective was to build a physically stable one-degree-of-freedom helicopter with accurate IMU-based angle estimation and a real-time PID controller, emphasis was placed on validating actual system behavior over purely static code verification. There were many instances where our connections were not connect so we were either not grounding some connections or not providing power to the PWM causing the motor to not work.

We divided testing into two complementary areas: **hardware verification** and **software/control verification**.

Hardware Verification

The oscilloscope and digital multimeter (DMM) were the primary hardware tools used. Early in the project, we verified the motor driver and optoisolator circuit on the PCB (not breadboard) and used the DMM to check supply voltages, continuity, and expected voltage drops. The oscilloscope was extensively employed to:

- Measure PWM drive signals to the optoisolator, ensuring the filtered motor command matched the intended duty cycle with acceptable ripple.
- Time the control interrupt service routine (ISR) by toggling a diagnostic GPIO high on ISR entry and low on exit, verifying ISR execution time was well within the 1 kHz control period.
- Observe system responses to step changes in target angle by plotting measured angle and low-pass filtered motor command simultaneously, enabling measurement of rise time, overshoot, and settling behavior.
- Verify the integrity of our custom soldered PCB by thoroughly inspecting all solder joints and connections. We used the DMM to confirm electrical continuity and absence of shorts, ensuring that every trace and component was correctly connected and operational before integration with the microcontroller.
- A significant issue we encountered was a loose wiring connection between the PWM output and the motor power supply. Each time we rebuilt or adjusted the system, we rigorously verified the PWM signal integrity using the oscilloscope to ensure it was correctly reaching the motor driver. This continuous monitoring was essential to rule out wiring faults as a cause of erratic/no motor behavior.

Software and Control Verification

Software testing was performed incrementally on the assembled system so that each modification could be directly evaluated through hardware behavior. The controller development and verification followed these steps:

1. **Open-loop motor characterization:** Before closed-loop control, we commanded a range of PWM duty cycles and observed motor and arm behavior to understand thrust characteristics and identify mechanical friction or binding. This informed safe control output limits and initial PID gain selection.
2. **Incremental PID integration:** Closed-loop control was added progressively:
 - *Proportional only (P):* Enabled proportional feedback first to verify system stability and assess steady-state error and responsiveness. Step responses to setpoint changes were recorded via oscilloscope.
 - *Proportional plus Derivative (PD):* Added derivative action to reduce overshoot and improve transient damping. The derivative gain was tuned by observing peak response and ringing in real time.

- *Proportional, Derivative, and Integral (PID)*: Finally, integral action was incorporated to eliminate steady-state error. Anti-windup was enabled, and long-term responses were monitored to ensure stable correction of bias without oscillations.
3. **Real-time tuning and observation:** The serial monitor was essential for live debugging and tuning. We printed diagnostics including estimated angle, raw accelerometer and gyroscope data, complementary filter state, and PID terms (P, I, D) along with the filtered motor command. This enabled:
 - On-the-fly adjustment of PID gains with immediate observation of system response.
 - Detection and correction of sign and scale errors in IMU data processing, such as inverted gyro readings that caused opposing control actions.
 - Monitoring of gyro bias estimates by computing resting averages at startup and verifying bias removal through stationary angle observations.
 4. **Complementary filter and sensor validation:** Static and dynamic tests were performed to validate sensor fusion. The arm was held at known angles to compare accelerometer-only angle versus the filtered estimate, and the assembly was rotated to verify gyro integration behavior. These tests helped uncover sign and scaling issues flagged by AI-assisted debugging, which were then corrected.
 5. **Automated sequences and functional tests:** The button-triggered demo sequence (vertical hang (-90°) $\rightarrow$ horizontal (0°) $\rightarrow +30^\circ \rightarrow -30^\circ \rightarrow$ horizontal(0°)) was executed and recorded while logging VGA and oscilloscope traces. Settling times and stable operating ranges for the tuned gains were measured. All features operated continuously without requiring resets during formal demonstrations.
 6. **Noise and robustness checks:** We analyzed noise levels on the control signal and angle estimate. When excessive noise was observed, we manipulated small digital low-pass filters on the motor command and re-ran step response tests to ensure control quality was maintained. Another crucial aspect of this lab was maintaining an organized and tidy setup, as electrical noise generated by the motors and circuitry could introduce interference. We ensured that wiring was kept neat and carefully routed to minimize crosstalk and electromagnetic interference, thereby preventing unexpected behavior caused by noise.

In summary, a combination of bench-level hardware verification and iterative software validation, which involved progressively adding control terms from proportional to full PID control while utilizing real-time logging and measurement tools, enabled us to identify and correct sensor processing errors, confirm timing constraints, and tune PID gains to achieve stable and repeatable system performance.

5 Results

The helicopter demonstrated stable control across the full range of setpoints from $[-80^\circ, +80^\circ]$. Most angle transitions were completed within approximately 2 seconds, highlighting the system’s fast response. Steady-state error was minimal due to the integral term in our PID controller, with measured angles consistently staying within a $\pm 0.6^\circ$ tolerance of the target for small transitions and within $\pm 0.6^\circ$ to $\pm 0.6 - 0.7^\circ$ for larger transitions, indicating high accuracy throughout the operating range.

Initially, commanding large upward transitions from the low-hanging position (-80°) to angles above horizontal caused the motor to trigger overcurrent protection due to the required transient current. This limitation prevented rapid full-range motion and reduced reliability for large-angle maneuvers. By carefully adjusting PID gains, we moderated the control output, allowing smooth acceleration and preventing overcurrent trips. After tuning, the system reliably transitioned between any two angles within the full range, including from -80° (low-hanging) to $+80^\circ$ (fully upright), without sacrificing accuracy or stability.

Mechanically, the hinge exhibited slight lateral play, introducing minor inconsistencies in motion. Attempts to tighten the hinge did not fully eliminate this movement. A more rigid hinge would likely have provided more consistent torque transmission, enabling finer PID tuning and faster settling times. Despite this, the helicopter achieved our primary objective: rapid, accurate, and stable angle control across the entire operating range.

Overall, the system’s performance illustrates the effectiveness of combining a PID controller with complementary-filtered IMU feedback. Fast response (2 s settling), high accuracy (errors within $\pm 0.6^\circ$), and reliable operation across the full range demonstrate the robustness of the implemented control system, while also highlighting the practical constraints imposed by mechanical play and electrical limitations.

AI Assistance

During the development and documentation of this lab, AI tools were employed selectively to improve both code robustness and report clarity. In the implementation phase, AI assisted in debugging sign and scaling errors when processing gyroscope and accelerometer data, which influenced the accuracy of the complementary filter and overall system response. It also provided guidance in diagnosing I²C communication issues, refining complementary filter parameters, and resolving timing inconsistencies between the PID interrupt service routine and PWM motor control. Additionally, AI suggested strategies to enhance numerical stability in fixed-point arithmetic and optimize loop execution for maintaining a consistent 1 kHz control rate.

During the documentation phase, AI was used to refine technical explanations, improve section organization, and ensure clarity in describing the theoretical background, control design, and experimental results. It also assisted in improving the precision and readability of figures, captions, and result interpretations while maintaining technical accuracy and alignment with the team’s original analysis. All AI-generated suggestions were critically evaluated and incorporated only when they supported the project’s educational goals; all design, tuning, and analytical decisions were made independently by the team.

To enhance our data collection, generative AI was used to create two python scripts - one for logging data from a serial port (to avoid overflowing the terminal buffer), and another to parse the insanely large CSV file generated by the first script and calculate precise settling times within a 6% tolerance. Generative AI was used for this purpose to enhance the analysis section of this report, not to demonstrate our technical skills with python. To this extent, we take no credit for the *python* scripts written for data collection and analysis.

Conclusion

This lab provided extensive hands-on experience with real-time embedded systems, emphasizing the challenges of maintaining precise timing, handling concurrent tasks, and integrating hardware with software in control applications. Implementing a PID controller for the helicopter arm allowed us to directly observe how sensor fusion, specifically through complementary filtering, transforms raw accelerometer and gyroscope data into reliable angle estimates critical for closed-loop control. By incrementally integrating the proportional, derivative, and integral terms, we gained insight into the trade-offs between responsiveness, stability, and overshoot, and the careful tuning required to achieve smooth, accurate motion without oscillations.

Working with the RP2040 microcontroller and its 1 kHz interrupt service routine reinforced the importance of efficient computation and deterministic timing. Even small errors, such as incorrect sign handling or scaling of IMU readings, significantly affected system behavior, motivating careful code design, fixed-point arithmetic considerations, and rigorous debugging. Iterative testing using serial monitoring, oscilloscopes, and logic analyzers was essential to validate both software and hardware performance, enabling real-time observation of angle tracking, control outputs, and PID term behavior.

Hardware considerations were equally critical. Maintaining organized wiring, minimizing noise, and electrically isolating the motor drive circuit using optoisolators ensured reliable operation and prevented disturbances from propagating into the microcontroller logic. Attention to mechanical design, such as rigid mounting of the arm and careful sensor placement near the pivot, reduced measurement errors from translational acceleration and ensured accurate feedback for the controller. Signal filtering, both digital and analog, helped suppress high-frequency noise that could destabilize the system.

This lab also highlighted practical aspects of embedded system design, including debouncing user input, managing setpoint sequences, and safely handling motor inductive spikes. Anti-windup strategies for the integral term and limits on control output demonstrated how theoretical control principles must be adapted for real-world hardware limitations. Through systematic testing and iterative tuning, we verified stable control across a wide range of setpoints, achieving rapid, precise, and repeatable arm motion.

Looking forward, improvements could include a more robust PCB layout to reduce noise susceptibility, expanded use of fixed-point arithmetic for ISR efficiency, and advanced sensor fusion or adaptive PID techniques to enhance system robustness under varying conditions. Overall, this lab significantly strengthened our understanding of embedded control systems, sensor integration, real-time programming, and hardware-software co-design, while reinforcing the importance of careful testing, debugging, and system-level thinking in developing

reliable, responsive electromechanical control systems.

6 AI Prompt Log

1. **Time/Date:** 2025-11-5 08:10
User Prompt / Request: Asked for C code snippet to read a character from serial input and adjust PID parameter accordingly; only proportional (p) supported.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 0
2. **Time/Date:** 2025-11-5 08:14
User Prompt / Request: Requested removing nested if statement inside the 'p' case to flatten logic for readability.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 1
3. **Time/Date:** 2025-11-5 08:16
User Prompt / Request: Asked to remove the quit ('q') feature so the loop runs indefinitely without exit option.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 2
4. **Time/Date:** 2025-11-5 08:20
User Prompt / Request: Requested detailed line-by-line explanation of the code to facilitate extending it to other parameters.
Suggested Edits: 0
Code Insertions: 0
Code Deletions / Modifications: 0
5. **Time/Date:** 2025-11-5 08:25
User Prompt / Request: Asked for a prompt log of the conversation including counts of edits and code changes accepted.
Suggested Edits: 0
Code Insertions: 0
Code Deletions / Modifications: 0
6. **Time/Date:** 2025-11-5 08:27
User Prompt / Request: Requested a more comprehensive prompt log with detailed context and explanations.
Suggested Edits: 0
Code Insertions: 0
Code Deletions / Modifications: 0

7. **Time/Date:** 2025-11-12 13:58
User Prompt / Request: Requested a Python script to listen to a serial port and log incoming data to a text file.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 0
8. **Time/Date:** 2025-11-12 14:03
User Prompt / Request: Asked to modify the logger script to include timestamps for each serial line entry.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 1
9. **Time/Date:** 2025-11-12 14:09
User Prompt / Request: Requested that the script automatically reconnect to the serial port if the connection is lost.
Suggested Edits: 1
Code Insertions: 2
Code Deletions / Modifications: 1
10. **Time/Date:** 2025-11-12 14:20
User Prompt / Request: Asked for a more memory-efficient implementation using buffered writes instead of writing each line individually.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 1
11. **Time/Date:** 2025-11-12 14:28
User Prompt / Request: Requested to make the serial port name and baud rate configurable via command-line arguments.
Suggested Edits: 1
Code Insertions: 2
Code Deletions / Modifications: 1
12. **Time/Date:** 2025-11-12 14:45
User Prompt / Request: Asked for the addition of simple console feedback to indicate when logging starts, stops, and reconnects.
Suggested Edits: 1
Code Insertions: 1
Code Deletions / Modifications: 0
13. **Time/Date:** 2025-11-12 15:10
User Prompt / Request: Requested a Python script to analyze the logged data and compute settling times for PID transitions, using tolerance-based evaluation.
Suggested Edits: 1

Code Insertions: 1
Code Deletions / Modifications: 0

14. **Time/Date:** 2025-11-12 15:15

User Prompt / Request: Asked for the settling time script to automatically detect setpoint changes and report settling statistics for each transition.

Suggested Edits: 1

Code Insertions: 2

Code Deletions / Modifications: 1

15. **Time/Date:** 2025-11-12 15:20

User Prompt / Request: Requested formatted output summarizing all transitions and tolerances, including delta magnitudes and time intervals.

Suggested Edits: 1

Code Insertions: 1

Code Deletions / Modifications: 1

16. **Time/Date:** 2025-11-12 15:25

User Prompt / Request: Asked for example output formatting to match report figures, showing each transition and corresponding settling band.

Suggested Edits: 1

Code Insertions: 1

Code Deletions / Modifications: 0

Notes:

- **Edits** refers to user requests for code or logic improvements.
- **Insertions** indicate new code snippets added.
- **Deletions** indicate removal of code features (e.g., quit option).
- **Modifications** are structural or logical changes to existing code.
- **Explanations** refer to detailed textual clarifications without code changes.

Works Cited

- Adams, V. Hunter. “*Complementary Filters.*” *V. Hunter Adams – Pico*. Available at: https://vanhunteradams.com/Pico/ReactionWheel/Complementary_Filters.html
- Adams, V. Hunter. “*Interfacing a 3×4 Matrix Keypad with RP2040.*” *V. Hunter Adams – Pico*. Available at: <https://vanhunteradams.com/Pico/Keypad/Keypad.html>
- Adams, V. Hunter. “*DC Motor Circuit and PWM on RP2040.*” *V. Hunter Adams – Pico*. Available at: <https://vanhunteradams.com/Pico/Helicopter/MotorCircuit.html>
- Adams, V. Hunter. “*Mechanical Construction of Lever Arm.*” *V. Hunter Adams – Pico*. Available at: https://vanhunteradams.com/Pico/Helicopter/Fan_Arm.html

7 Code Listing

Listing 2: helicopter.c code listing

```
1  /**
2
3  * V. Hunter Adams (vha3@cornell.edu)
4  *
5  * This demonstration utilizes the MPU6050.
6  * It gathers raw accelerometer/gyro measurements, scales
7  * them, and plots them to the VGA display. The top plot
8  * shows gyro measurements, bottom plot shows accelerometer
9  * measurements.
10 *
11 * HARDWARE CONNECTIONS
12 * - GPIO 16 ---> VGA Hsync
13 * - GPIO 17 ---> VGA Vsync
14 * - GPIO 18 ---> 470 ohm resistor ---> VGA Green
15 * - GPIO 19 ---> 330 ohm resistor ---> VGA Green
16 * - GPIO 20 ---> 330 ohm resistor ---> VGA Blue
17 * - GPIO 21 ---> 330 ohm resistor ---> VGA Red
18 * - RP2040 GND ---> VGA GND
19 * - GPIO 8 ---> MPU6050 SDA
20 * - GPIO 9 ---> MPU6050 SCL
21 * - 3.3v ---> MPU6050 VCC
22 * - RP2040 GND ---> MPU6050 GND
23 */
24
25
26 // Include standard libraries
27 #include <stdio.h>
28 #include <stdlib.h>
29 #include <math.h>
30 #include <string.h>
31 // Include PICO libraries
32 #include "pico/stdlib.h"
33 #include "pico/multicore.h"
34 // Include hardware libraries
35 #include "hardware/pwm.h"
36 #include "hardware/dma.h"
37 #include "hardware/irq.h"
38 #include "hardware/adc.h"
39 #include "hardware/pio.h"
40 #include "hardware/i2c.h"
41 #include "hardware/clocks.h"
42 // Include custom libraries
43 #include "vga16_graphics_v2.h"
44 #include "mpu6050.h"
45 #include "pt_cornell_rp2040_v1_4.h"
46
47 // Arrays in which raw measurements will be stored
48 fix15 acceleration[3], gyro[3];
49 fix15 accel_angle, gyro_angle_delta;
50 fix15 complementary_angle;
```

```

51 fix15 filter_acc_z, filter_acc_y;
52
53 // character array
54 char screentext[40];
55
56 // --- Sequence debug/status ---
57 static volatile bool      seq_active   = false;
58 static volatile int       seq_phase    = -1;      // -1=idle, 0,1,2,3 =
           steps
59 static volatile uint32_t seq_start_ms = 0;
60
61 // helper to print sequence phase names
62 static inline const char* seq_phase_name(int ph) {
63     switch (ph) {
64         case -1: return "idle";
65         case 0: return "t=0 -> 0  ";
66         case 1: return "t=5 -> +30 ";
67         case 2: return "t=10 -> -30 ";
68         case 3: return "t=15 -> 0  ";
69         default: return "?";
70     }
71 }
72
73 // draw speed
74 int threshold = 50 ;
75
76 // Some macros for max/min/abs
77 #define min(a,b) ((a<b) ? a:b)
78 #define max(a,b) ((a<b) ? b:a)
79 #define abs(a) ((a>0) ? a:-a)
80
81
82 // semaphore
83 static struct pt_sem vga_semaphore ;
84
85 fix15 desired_angle;
86
87 // Some paramters for PWM
88 #define WRAPVAL 5000
89 #define CLKDIV 25.0
90 #define PWM_OUT 4
91
92 #define PI float2fix15(3.14159265358979)
93
94 uint slice_num ;
95
96 // BUTTON
97 #define BUTTON_PIN 15
98 #define DEBOUNCE_SAMPLES 4
99
100 static struct pt_sem sequence_semaphore;
101 volatile bool motor_enabled = true;      // motor off when holding button
102 volatile bool sequence_running = false; // ignore re-triggers while
           running

```

```

103 // helper for degrees      fix15
104 static inline fix15 deg15(float deg) { return float2fix15(deg); }
105
106 // Polled button with your FSM/debounce; active-LOW button
107 static PT_THREAD (protothread_button(struct pt *pt))
108 {
109     PT_BEGIN(pt);
110
111
112     static int STATE_1 = 0;          // 0:not pressed, 1:maybe, 2:pressed, 3:
113         maybe rel
114     static int possible = 1;
115     static int stable = 1;
116     static int same_cnt = 0;
117
118     while (1) {
119         int i = gpio_get(BUTTON_PIN);    // 0=pressed, 1=released
120
121         switch (STATE_1) {
122
123             case 0: // not pressed
124                 if (i != 1) {                // went LOW? maybe pressed
125                     possible = i;            // 0
126                     same_cnt = 1;
127                     STATE_1 = 1;
128                 }
129                 break;
130
131             case 1: // maybe pressed
132                 if (i == possible) {
133                     if (++same_cnt >= DEBOUNCE_SAMPLES) {
134                         // PRESSED confirmed
135                         stable = possible;
136                         motor_enabled = false;
137                         printf("[BTN] pressed -> motor OFF\n");
138
139                         STATE_1 = 2;
140                         same_cnt = 0;
141                     }
142                 } else {                    // bounced back high
143                     STATE_1 = 0;
144                     same_cnt = 0;
145                 }
146                 break;
147
148             case 2: // pressed (stable low)
149                 if (i != 0) {                // maybe released?
150                     possible = i;            // 1
151                     same_cnt = 1;
152                     STATE_1 = 3;
153                 }
154                 break;
155

```



```

156     case 3: // maybe released
157         if (i == possible) {
158             if (++same_cnt >= DEBOUNCE_SAMPLES) {
159                 stable = possible;
160                 motor_enabled = true;
161                 printf("[BTN] pressed -> motor OFF\n");
162                 if (!sequence_running) {
163                     sequence_running = true;
164                     PT_SEM_SIGNAL(pt, &sequence_semaphore);
165                 }
166
167                 STATE_1 = 0;                // back to idle
168                 same_cnt = 0;
169             }
170             else {                          // bounced low again
171                 STATE_1 = 2;
172                 same_cnt = 0;
173             }
174             break;
175         }
176
177         PT_YIELD_usec(1000);
178     }
179
180     PT_END(pt);
181 }
182
183 // Sequence thread
184 // Adjusts the target angle through a time sequence
185 static PT_THREAD (protothread_sequence(struct pt *pt))
186 {
187     PT_BEGIN(pt);
188     while (true) {
189         PT_SEM_WAIT(pt, &sequence_semaphore);
190
191         // Mark start for timing/printing
192         seq_active = true;
193         seq_phase = 0;
194         seq_start_ms = to_ms_since_boot(get_absolute_time());
195
196         // t = 0 s -> 0
197         desired_angle = deg15(0.0f);
198         PT_YIELD_usec(5000000);
199
200         // t = 5 s -> +30
201         seq_phase = 1;
202         desired_angle = deg15(+30.0f);
203
204         PT_YIELD_usec(5000000);
205
206         // t = 10 s -> -30
207         seq_phase = 2;
208         desired_angle = deg15(-30.0f);
209

```

```

210     PT_YIELD_usec(5000000);
211
212     // t = 15 s -> 0
213     seq_phase = 3;
214     desired_angle = deg15(0.0f);
215
216     // Done
217     seq_active = false;
218     seq_phase = -1;
219     sequence_running = false;
220     printf("[SEQ] complete\n");
221 }
222 PT_END(pt);
223 }
224
225 // PWM duty cycle
226 volatile int control ;
227 volatile int old_control ;
228 fix15 control_pix;
229
230 // Combines gyro and accel data to allow stabl angle estimation
231 void complementary_filter()
232 {
233     filter_acc_y = filter_acc_y + ((acceleration[1] - filter_acc_y)>>4);
234     filter_acc_z = filter_acc_z + ((acceleration[2] - filter_acc_z)>>4);
235
236     // accel_angle = multfix15(divfix(acceleration[0], acceleration[1]),
237     //                          oneeightyoverpi);
238     accel_angle = multfix15(float2fix15(atan2(-filter_acc_y, filter_acc_z)
239     ), oneeightyoverpi);
240     gyro_angle_delta = multfix15(gyro[0], zeropt001) ;
241
242     complementary_angle = multfix15(complementary_angle - gyro_angle_delta
243     , zeropt999) + multfix15(accel_angle, zeropt001);
244 }
245
246 // PID Controller
247 struct PID
248 {
249     fix15 Kp;
250     fix15 Ki;
251     fix15 Kd; // kevin durant Easymoneysniper (our fav player + PID term)
252     fix15 prev_err;
253     fix15 integral;
254     fix15 max_integral;
255     int max;
256     int min;
257 };
258
259 // Initial PID paramters
260 struct PID pid = {float2fix15(200), float2fix15(1.0), float2fix15(75), 0,
261     0, int2fix15(2800), 2800, 100};
262
263 fix15 err;

```

```

260
261 // Perform one PID step
262 void pid_step()
263 {
264     err = (desired_angle - complementary_angle);
265
266     pid.integral = (pid.integral + err) < pid.max_integral ? pid.integral
267                   + err : pid.max_integral; // clamp integral accumulation
268     pid.integral = pid.integral > 0 ? pid.integral : 0;
269     pid.prev_err = err;
270
271     fix15 derivative_term = multfix15(gyro[0], pid.Kd);
272
273     int temp_control = fix2int15(multfix15(err, pid.Kp) + multfix15(pid.
274                               integral, pid.Ki) + derivative_term);
275
276     if (temp_control > pid.max) {
277         control = pid.max;
278     }
279     else if (temp_control < pid.min) {
280         control = pid.min;
281     }
282     else {
283         control = temp_control;
284     }
285
286     if (!motor_enabled) control = 0;
287
288     if (control != old_control) {
289         old_control = control;
290         pwm_set_chan_level(slice_num, PWM_CHAN_A, control);
291     }
292 }
293
294 // Reads IMU, updates filter and PID, then signals VGA
295 // Interrupt service routine
296 void on_pwm_wrap() {
297
298     // Clear the interrupt flag that brought us here
299     pwm_clear_irq(pwm_gpio_to_slice_num(5));
300
301     // Read the IMU
302     // NOTE! This is in 15.16 fixed point. Accel in g's, gyro in deg/s
303     // If you want these values in floating point, call fix2float15() on
304     // the raw measurements.
305     // m = motor_disp + ((pwm_on_time - motor_disp)>>6)
306     mpu6050_read_raw(acceleration, gyro);
307
308     complementary_filter();
309     pid_step();
310
311     // Signal VGA to draw
312     PT_SEM_SIGNAL(pt, &vga_semaphore);

```

```

312 }
313
314 // Plots angles and control signals on the VGA display
315 // Thread that draws to VGA display
316 static PT_THREAD (protothread_vga(struct pt *pt))
317 {
318     // Indicate start of thread
319     PT_BEGIN(pt) ;
320
321     // We will start drawing at column 81
322     static int xcoord = 81 ;
323
324     // Rescale the measurements for display
325     static float OldRange = 180. ; // (+/- 90)
326     static float NewRange = 150. ; // (looks nice on VGA)
327     static float OldMin = -90. ;
328     static float OldMax = 90. ;
329
330     // Control rate of drawing
331     static int throttle ;
332
333     // Draw the static aspects of the display
334     setTextSize(1) ;
335     setTextColor(WHITE);
336
337     // Draw bottom plot
338     drawHLine(75, 430, 5, CYAN) ;
339     drawHLine(75, 355, 5, CYAN) ;
340     drawHLine(75, 280, 5, CYAN) ;
341     drawVLine(80, 280, 150, CYAN) ;
342     sprintf(screentext, "+1400" ) ;
343     setCursor(50, 350) ;
344     writeString(screentext) ;
345     sprintf(screentext, "+2800" ) ;
346     setCursor(50, 280) ;
347     writeString(screentext) ;
348     sprintf(screentext, "0" ) ;
349     setCursor(50, 425) ;
350     writeString(screentext) ;
351
352     // Draw top plot
353     drawHLine(75, 230, 5, CYAN) ;
354     drawHLine(75, 155, 5, CYAN) ;
355     drawHLine(75, 80, 5, CYAN) ;
356     drawHLine(75, 180, 5, CYAN) ;
357     drawHLine(75, 130, 5, CYAN) ;
358     drawVLine(80, 80, 150, CYAN) ;
359
360     // Prints the angles
361     sprintf(screentext, "0" ) ;
362     setCursor(50, 150) ;
363     writeString(screentext) ;
364
365     sprintf(screentext, "+30" ) ;

```

```

366     setCursor(50, 125) ;
367     writeString(screentext) ;
368
369     sprintf(screentext, "-30" ) ;
370     setCursor(50, 175) ;
371     writeString(screentext) ;
372
373     sprintf(screentext, "+90" ) ;
374     setCursor(45, 75) ;
375     writeString(screentext) ;
376
377     sprintf(screentext, "-90" ) ;
378     setCursor(45, 225) ;
379     writeString(screentext) ;
380
381     while (true) {
382         // Wait on semaphore
383         PT_SEM_WAIT(pt, &vga_semaphore);
384         // Increment drawspeed controller
385         throttle += 1 ;
386         // If the controller has exceeded a threshold, draw
387         if (throttle >= threshold) {
388             // Zero drawspeed controller
389             throttle = 0 ;
390
391             // Erase a column
392             drawVLine(xcoord, 0, 480, BLACK) ;
393
394             // Draw bottom plot (multiply by 120 to scale from +/-2 to
395                 // +/-250)
396
397             control_pix = control_pix + ((int2fix15(control) - control_pix
398                 ) >> 5);
399             drawPixel(xcoord, 505 - (int)(NewRange*((float)((fix2float15(
400                 control_pix) / 15.55)-OldMin)/OldRange)), GREEN) ;
401
402             // Draw top plot
403             drawPixel(xcoord, 230 - (int)(NewRange*((float)((fix2float15(
404                 desired_angle)) - OldMin)/OldRange)), GREEN) ;
405             drawPixel(xcoord, 230 - (int)(NewRange*((float)((fix2float15(
406                 complementary_angle))-OldMin)/OldRange)), BLUE) ;
407
408             // Update horizontal cursor
409             if (xcoord < 609) {
410                 xcoord += 1 ;
411             }
412             else {
413                 xcoord = 81 ;
414             }
415             fillRect(10, 445, 620, 30, BLACK);
416             setCursor(10, 460);
417             setTextColor(WHITE);
418             sprintf(screentext, "PID: Kp=%5.1f Ki=%5.2f Kd=%5.1f",

```

```

414         fix2float15(pid.Kp), fix2float15(pid.Ki), fix2float15(pid.Kd))
415         ;
416         writeString(screentext);
417     }
418     // Indicate end of thread
419     PT_END(pt);
420 }
421
422 // User input thread. User can change draw speed
423 static PT_THREAD (protothread_serial(struct pt *pt))
424 {
425     PT_BEGIN(pt) ;
426     static char classifier ;
427     static int test_in ;
428     static float float_in ;
429     while(1) {
430
431
432         printf("error: %f \n", fix2float15(desired_angle -
433             complementary_angle));
434         printf("control: %d \n", (control));
435
436         ##### BEGIN AI-GENERATED CODE #####
437         printf("enter p/i/d/a: ");
438         serial_read;
439         sscanf(pt_serial_in_buffer, "%c", &classifier);
440
441         if (classifier == 'p') {
442             sprintf(pt_serial_out_buffer, "Set Kp: ");
443             serial_write;
444             serial_read;
445             sscanf(pt_serial_in_buffer, "%f", &float_in);
446             pid.Kp = float2fix15(float_in);
447         }
448         ##### END AI-GENERATED CODE #####
449         // adjust Ki term in serial
450         else if (classifier == 'i') {
451             sprintf(pt_serial_out_buffer, "Set Ki: ");
452             serial_write;
453             serial_read;
454             sscanf(pt_serial_in_buffer, "%f", &float_in);
455             pid.Ki = float2fix15(float_in);
456         }
457
458         // adjust Kd term in serial
459         else if (classifier == 'd') {
460             sprintf(pt_serial_out_buffer, "Set Kd: ");
461             serial_write;
462             serial_read;
463             sscanf(pt_serial_in_buffer, "%f", &float_in);
464             pid.Kd = float2fix15(float_in);
465         }

```

```

466         // adjust desired_angle term in serial
467         else if (classifier == 'a') {
468             sprintf(pt_serial_out_buffer, "Set Desired Angle: ");
469             serial_write;
470             serial_read;
471             sscanf(pt_serial_in_buffer, "%f", &float_in);
472             desired_angle = float2fix15(float_in);
473         }
474     }
475
476     PT_END(pt) ;
477 }
478
479 // Entry pMoint for core 1
480 void core1_entry() {
481     pt_add_thread(protothread_vga) ;
482     pt_schedule_start ;
483 }
484
485
486 int main() {
487
488     // semaphores
489     PT_SEM_INIT(&vga_semaphore, 0);
490     PT_SEM_INIT(&sequence_semaphore, 0);
491
492     // Overclock
493     set_sys_clock_khz(150000, true) ;
494
495     // Initialize stdio
496     stdio_init_all();
497
498     // Initialize VGA
499     initVGA() ;
500
501     // button GPIO
502     gpio_init(BUTTON_PIN);
503     gpio_set_dir(BUTTON_PIN, GPIO_IN);
504     gpio_pull_up(BUTTON_PIN); // active-LOW with pull-up
505
506     //
507     ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
508     // I2C CONFIGURATION
509     ////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
510     i2c_init(I2C_CHAN, I2C_BAUD_RATE) ;
511     gpio_set_function(SDA_PIN, GPIO_FUNC_I2C) ;
512     gpio_set_function(SCL_PIN, GPIO_FUNC_I2C) ;
513
514     // Pullup resistors on breakout board, don't need to turn on internals
515     // gpio_pull_up(SDA_PIN) ;
516     // gpio_pull_up(SCL_PIN) ;
517
518     // MPU6050 initialization

```

```

517 mpu6050_reset();
518 mpu6050_read_raw(acceleration, gyro);
519
520 //
521 ////////////////////////////////////////////////////
522 ////////////////////////////////////////////////////
523 ////////////////////////////////////////////////////
524 ////////////////////////////////////////////////////
525 ////////////////////////////////////////////////////
526 ////////////////////////////////////////////////////
527 ////////////////////////////////////////////////////
528 ////////////////////////////////////////////////////
529 ////////////////////////////////////////////////////
530 ////////////////////////////////////////////////////
531 // Tell GPIO's 4,5 that they allocated to the PWM
532 gpio_set_function(5, GPIO_FUNC_PWM);
533 gpio_set_function(4, GPIO_FUNC_PWM);
534
535 ////////////////////////////////////////////////////
536 ////////////////////////////////////////////////////
537 ////////////////////////////////////////////////////
538 ////////////////////////////////////////////////////
539 ////////////////////////////////////////////////////
540 ////////////////////////////////////////////////////
541 ////////////////////////////////////////////////////
542 ////////////////////////////////////////////////////
543 // Tell GPIO PWM_OUT that it is allocated to the PWM
544 gpio_set_function(PWM_OUT, GPIO_FUNC_PWM);
545
546 // Find out which PWM slice is connected to GPIO 5 (it's slice 2, same
547 // for 4)
548 slice_num = pwm_gpio_to_slice_num(5);
549
550 // Mask our slice's IRQ output into the PWM block's single interrupt
551 // line,
552 // and register our interrupt handler
553 pwm_clear_irq(slice_num);
554 pwm_set_irq_enabled(slice_num, true);
555 irq_set_exclusive_handler(PWM_IRQ_WRAP, on_pwm_wrap);
556 irq_set_enabled(PWM_IRQ_WRAP, true);
557
558 // This section configures the period of the PWM signals
559 pwm_set_wrap(slice_num, WRAPVAL) ;
560 pwm_set_clkdiv(slice_num, CLKDIV) ;
561
562 // This sets duty cycle
563 pwm_set_chan_level(slice_num, PWM_CHAN_B, 0);
564 pwm_set_chan_level(slice_num, PWM_CHAN_A, 0);
565
566 // Start the channel
567 pwm_set_mask_enabled((1u << slice_num));
568
569 desired_angle = float2fix15(0.0f);
570
571 //
572 ////////////////////////////////////////////////////

```



```

558  //////////////////////////////////// ROCK AND ROLL
559  ////////////////////////////////////
560  //
561  ////////////////////////////////////
562  // start core 1
563  multicore_reset_core1();
564  multicore_launch_core1(core1_entry);
565  // start core 0
566  pt_add_thread(protothread_serial) ;
567  pt_add_thread(protothread_sequence);
568  pt_add_thread(protothread_button);
569  pt_schedule_start ;
570 }

```